RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence and Data Science

V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 5: Information Extraction using spaCy

Name: Praveen L
Register Number: 241801213
Dataset: Amazon Fine Food Reviews (Kaggle)

Date: 02-10-2026
Tools: Python, Pandas, spaCy (en_core_web_sm), Matplotlib

Aim

To process customer reviews using the spaCy NLP library and build an information extraction system that identifies tokens, lemmas, part-of-speech tags and named entities, and organises the extracted information into a structured format.

Task 1: Load the dataset and import the required libraries

In [1]:

```
# Install once (in a terminal or notebook cell):
#   pip install spacy pandas matplotlib
#   python -m spacy download en_core_web_sm

import re
from collections import Counter

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import spacy

sns.set_theme(style='whitegrid')
pd.set_option('display.max_colwidth', 110)

nlp = spacy.load('en_core_web_sm')       # small English pipeline
print("spaCy version :", spacy.__version__)
print("Model loaded  :", f"{nlp.meta['lang']}_{nlp.meta['name']}", nlp.meta['version'])
print("Pipeline      :", nlp.pipe_names)
```

Out [1]:

```
spaCy version : 3.8.16
Model loaded  : en_core_web_sm 3.8.0
Pipeline      : ['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
```

In [2]:

```
df = pd.read_csv('Reviews.csv')
print("Dataset loaded successfully")
print("Shape:", df.shape)
df[['Id', 'ProductId', 'Score', 'Summary', 'Text']].head()
```

Out [2]:

```
Dataset loaded successfully
Shape: (10112, 10)
```

Columns 1–4 of 5

	Id	ProductId	Score	Summary
0	1	B001E4KFG0	5	Good Quality Dog Food
1	2	B00813GRG4	1	Not as Advertised
2	3	B000LQOCH0	4	"Delight" says it all
3	4	B000UA0QIQ	2	Cough Medicine
4	5	B006K2ZZ7K	5	Great taffy

Columns 5–5 of 5

	Text
0	I have bought several of the Vitality canned dog food products and have found them all to be of good qualit...
1	Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure ...
2	This is a confection that has been around a few centuries. It is a light, pillowy citrus gelatin with nuts...
3	If you are looking for the secret ingredient in Robitussin I believe I have found it. I got this in additi...
4	Great taffy at a great price. There was a wide assortment of yummy taffy. Delivery was very quick. If yo...

5 rows × 5 columns

> **Note:** As in Ex 4, Reviews.csv contains the first 10,112 reviews (Id 1 to 10,112) of the Kaggle Amazon Fine Food Reviews dataset.

Task 2: Clean the dataset

In [3]:

```
reviews = df[['Text']].copy()                       # select the Text column
print("Reviews selected              :", len(reviews))

print("Missing values in Text        :", reviews['Text'].isnull().sum())
reviews = reviews.dropna(subset=['Text'])           # remove missing values

print("Duplicate reviews             :", reviews['Text'].duplicated().sum())
reviews = reviews.drop_duplicates(subset=['Text'])  # remove duplicate reviews

reviews = reviews.head(10000).reset_index(drop=True)   # keep the first 10,000 reviews
print("Reviews retained for analysis :", len(reviews))

# Replace HTML line breaks such as <br /> with spaces so they are not treated as words
reviews['Text'] = (reviews['Text'].str.replace(r'<[^>]+>', ' ', regex=True)
                                  .str.replace(r'\s+', ' ', regex=True).str.strip())
```

Out [3]:

```
Reviews selected              : 10112
Missing values in Text        : 0
Duplicate reviews             : 487
Reviews retained for analysis : 9625
```

Text Preprocessing and Part-of-Speech (POS) Tags

Task 3: Apply the spaCy pipeline and display tokens, lemmas and POS tags for five reviews

In [4]:

```
# Process all reviews with the spaCy pipeline (tokenizer, tagger, parser, lemmatizer, NER)
docs = list(nlp.pipe(reviews['Text'], batch_size=256))
print("Reviews processed:", len(docs))
print("Total tokens     :", sum(len(doc) for doc in docs))
```

Out [4]:

```
Reviews processed: 9625
Total tokens     : 857146
```

In [5]:

```
# Choose five short reviews so the token tables are easy to read
lengths = pd.Series([len(doc) for doc in docs])
sample_ids = lengths[lengths.between(14, 24)].sample(5, random_state=3).index.tolist()

for n, i in enumerate(sample_ids, start=1):
    doc = docs[i]
    print(f"Review {n} (row {i}): {doc.text}")
    table = pd.DataFrame([[t.text for t in doc], [t.lemma_ for t in doc], [t.pos_ for t in doc]],
                         index=['Token', 'Lemma', 'POS'], columns=range(1, len(doc) + 1))
    display(table)
```

Out [5]:

Review 1 (row 205): Fresh scent and taste. Easy to eat one tin in few hours. I use them as favors and everyone loved them.

Columns 1–19 of 24

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Token	Fresh	scent	and	taste	.	Easy	to	eat	one	tin	in	few	hours	.	I	use	them	as	favors
Lemma	fresh	scent	and	taste	.	easy	to	eat	one	tin	in	few	hour	.	I	use	they	as	favor
POS	ADJ	NOUN	CCONJ	NOUN	PUNCT	ADJ	PART	VERB	NUM	NOUN	ADP	ADJ	NOUN	PUNCT	PRON	VERB	PRON	ADP	NOUN

Columns 20–24 of 24

	20	21	22	23	24
Token	and	everyone	loved	them	.
Lemma	and	everyone	love	they	.
POS	CCONJ	PRON	VERB	PRON	PUNCT

3 rows × 24 columns

Review 2 (row 1445): These dinners are as close to perfect as can be found here in the mountains. Quick, easy and delicious!

Columns 1–17 of 22

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Token	These	dinners	are	as	close	to	perfect	as	can	be	found	here	in	the	mountains	.	Quick
Lemma	these	dinner	be	as	close	to	perfect	as	can	be	find	here	in	the	mountain	.	quick
POS	DET	NOUN	AUX	ADV	ADJ	PART	VERB	SCONJ	AUX	AUX	VERB	ADV	ADP	DET	NOUN	PUNCT	ADJ

Columns 18–22 of 22

	18	19	20	21	22
Token	,	easy	and	delicious	!
Lemma	,	easy	and	delicious	!
POS	PUNCT	ADJ	CCONJ	ADJ	PUNCT

3 rows × 22 columns

Review 3 (row 3140): I am a bit disappointed. The flavor was not what I wanted or expected.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Token	I	am	a	bit	disappointed	.	The	flavor	was	not	what	I	wanted	or	expected	.
Lemma	I	be	a	bit	disappointed	.	the	flavor	be	not	what	I	want	or	expect	.
POS	PRON	AUX	DET	NOUN	ADJ	PUNCT	DET	NOUN	AUX	PART	PRON	PRON	VERB	CCONJ	VERB	PUNCT

3 rows × 16 columns

Review 4 (row 6053): love the contents, but two of eight containers were badly dented. jan

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Token	love	the	contents	,	but	two	of	eight	containers	were	badly	dented	.	jan
Lemma	love	the	content	,	but	two	of	eight	container	be	badly	dent	.	jan
POS	VERB	DET	NOUN	PUNCT	CCONJ	NUM	ADP	NUM	NOUN	AUX	ADV	VERB	PUNCT	PROPN

3 rows × 14 columns

Review 5 (row 9145): haven't used it yet , but if going by past Marshal Creek products ... ? should be good.

Columns 1–17 of 20

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Token	have	n't	used	it	yet	,	but	if	going	by	past	Marshal	Creek	products	...	?	should
Lemma	have	not	use	it	yet	,	but	if	go	by	past	Marshal	Creek	product	...	?	should
POS	AUX	PART	VERB	PRON	ADV	PUNCT	CCONJ	SCONJ	VERB	ADP	ADP	PROPN	PROPN	NOUN	PUNCT	PUNCT	AUX

Columns 18–20 of 20

	18	19	20
Token	be	good	.
Lemma	be	good	.
POS	AUX	ADJ	PUNCT

3 rows × 20 columns

In [6]:

```
# Meaning of the POS tags used above
pos_tags = sorted({t.pos_ for i in sample_ids for t in docs[i]})
meanings = [spacy.explain(p) for p in pos_tags]
pd.DataFrame({'POS Tag': pos_tags, 'Meaning': meanings}).set_index('POS Tag')
```

Out [6]:

POS Tag	Meaning
ADJ	adjective
ADP	adposition
ADV	adverb
AUX	auxiliary
CCONJ	coordinating conjunction
DET	determiner
NOUN	noun
NUM	numeral
PART	particle
PRON	pronoun
PROPN	proper noun
PUNCT	punctuation
SCONJ	subordinating conjunction
VERB	verb

14 rows × 1 columns

> **Observation:** spaCy splits text into tokens, including punctuation and contractions (*haven't* becomes *have* + *n't*, with the lemma *not*). Lemmas reduce words to their base form (*loved* → *love*, *hours* → *hour*, *were* → *be*, *containers* → *container*). The POS tags are mostly correct, with a few mistakes on informal text, for example *perfect* is tagged as a verb in review 2 and the signature *jan* as a proper noun in review 4.

Task 4: Identify the POS categories (Nouns, Verbs, Adjectives, Adverbs)

In [7]:

```
categories = {'Nouns': ['NOUN', 'PROPN'], 'Verbs': ['VERB'],
              'Adjectives': ['ADJ'], 'Adverbs': ['ADV']}

rows = []
for n, i in enumerate(sample_ids, start=1):
    row = {'Review': n}
    for name, tags in categories.items():
        row[name] = ', '.join(t.text for t in docs[i] if t.pos_ in tags) or '-'
    rows.append(row)

pd.DataFrame(rows).set_index('Review')
```

Out [7]:

Review	Nouns	Verbs	Adjectives	Adverbs
1	scent, taste, tin, hours, favors	eat, use, loved	Fresh, Easy, few	-
2	dinners, mountains	perfect, found	close, Quick, easy, delicious	as, here
3	bit, flavor	wanted, expected	disappointed	-
4	contents, containers, jan	love, dented	-	badly
5	Marshal, Creek, products	used, going	good	yet

5 rows × 4 columns

In [8]:

```
# Most frequent words (lemmas) in each POS category across all reviews
top_words = {}
for name, tags in categories.items():
    counts = Counter(t.lemma_.lower() for doc in docs for t in doc
                     if t.pos_ in tags and t.is_alpha and not t.is_stop)
    top_words[name] = [f"{word} ({count})" for word, count in counts.most_common(10)]

top_words_df = pd.DataFrame(top_words, index=range(1, 11))
top_words_df.index.name = 'Rank'
top_words_df
```

Out [8]:

Rank	Nouns	Verbs	Adjectives	Adverbs
1	flavor (3249)	try (2696)	good (4468)	definitely (504)
2	coffee (3169)	love (2646)	great (2731)	far (467)
3	product (3004)	buy (2516)	little (1333)	well (464)
4	taste (2129)	like (2229)	well (954)	pretty (438)
5	food (2056)	find (2227)	sweet (863)	actually (401)
6	cup (1790)	taste (2138)	hot (819)	highly (392)
7	tea (1790)	eat (1728)	free (797)	usually (358)
8	time (1634)	use (1406)	small (723)	long (342)
9	bag (1394)	think (1284)	easy (691)	probably (339)
10	price (1375)	order (1219)	nice (684)	instead (321)

10 rows × 4 columns

> **Observation:** The most common nouns are product-related (*flavor*, *coffee*, *product*, *taste*, *food*, *tea*), the most common adjectives are opinions (*good*, *great*, *sweet*, *easy*, *nice*) and the most common verbs describe buying and using (*try*, *love*, *buy*, *like*, *eat*, *order*). These words summarise what customers talk about.

Task 5: Frequency of the major POS categories in the reviews

In [9]:

```
pos_counts = Counter(t.pos_ for doc in docs for t in doc if not t.is_space)
pos_freq = pd.DataFrame(pos_counts.most_common(), columns=['POS Tag', 'Count'])
pos_freq['Meaning'] = pos_freq['POS Tag'].apply(spacy.explain)
pos_freq['Percentage (%)'] = (pos_freq['Count'] / pos_freq['Count'].sum() * 100).round(2)
pos_freq.set_index('POS Tag')
```

Out [9]:

POS Tag	Count	Meaning	Percentage (%)
NOUN	138369	noun	16.14
PRON	99354	pronoun	11.59
PUNCT	98668	punctuation	11.51
VERB	93252	verb	10.88
ADJ	70232	adjective	8.19
ADP	67153	adposition	7.83
DET	66817	determiner	7.80
AUX	55431	auxiliary	6.47
ADV	49685	adverb	5.80
CCONJ	31676	coordinating conjunction	3.70
PROPN	30823	proper noun	3.60
PART	24242	particle	2.83
SCONJ	16350	subordinating conjunction	1.91
NUM	11407	numeral	1.33
SYM	1661	symbol	0.19
INTJ	1268	interjection	0.15
X	758	other	0.09

17 rows × 3 columns

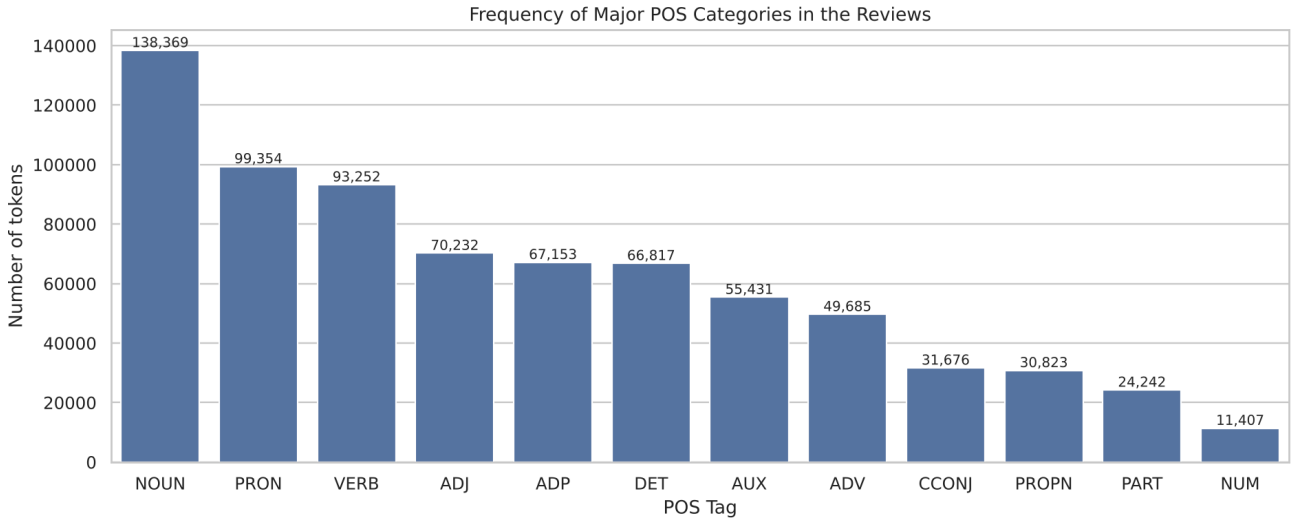
In [10]:

```
major = pos_freq[pos_freq['POS Tag'].isin(['NOUN', 'PROPN', 'VERB', 'ADJ', 'ADV', 'PRON', 'AUX',
                                           'ADP', 'DET', 'CCONJ', 'NUM', 'PART'])]

plt.figure(figsize=(12, 5))
ax = sns.barplot(data=major, x='POS Tag', y='Count', color='#4C72B0')
ax.bar_label(ax.containers[0], labels=[f"{v:,}" for v in major['Count']], fontsize=9)
plt.title('Frequency of Major POS Categories in the Reviews')
plt.ylabel('Number of tokens')
plt.tight_layout()
plt.show()

summary = {name: sum(pos_counts[t] for t in tags) for name, tags in categories.items()}
print("Main content-word categories:", summary)
```

Out [10]:




```
Main content-word categories: {'Nouns': 169192, 'Verbs': 93252, 'Adjectives': 70232, 'Adverbs': 49685}
```

> **Observation:** Nouns are the most frequent POS category (16.14% of all tokens), followed by pronouns (11.59%), punctuation (11.51%) and verbs (10.88%). Adjectives make up 8.19% of the tokens, which is high and shows that reviews are full of opinions and descriptions.

Named Entity Recognition (NER)

Task 6: Extract entities (PERSON, ORG, DATE, MONEY, QUANTITY, PRODUCT)

In [11]:

```
target_labels = ['PERSON', 'ORG', 'DATE', 'MONEY', 'QUANTITY', 'PRODUCT']

entity_rows = []
for i, doc in enumerate(docs):
    for ent in doc.ents:
        if ent.label_ in target_labels:
            entity_rows.append({'Review_ID': i, 'Entity': ent.text, 'Label': ent.label_,
                                'Description': spacy.explain(ent.label_)})

entities_df = pd.DataFrame(entity_rows)
n_reviews = entities_df['Review_ID'].nunique()
print("Entities extracted:", len(entities_df), "from", n_reviews, "reviews")
entities_df.head(15)
```

Out [11]:

Entities extracted: 15671 from 5842 reviews

	Review_ID	Entity	Label	Description
0	0	Vitality	ORG	Companies, agencies, institutions, etc.
1	0	My Labrador	PERSON	People, including fictional
2	1	Jumbo Salted	ORG	Companies, agencies, institutions, etc.
3	2	around a few centuries	DATE	Absolute or relative dates or periods
4	2	pillowy citrus gelatin	ORG	Companies, agencies, institutions, etc.
5	2	C.S. Lewis'	ORG	Companies, agencies, institutions, etc.
6	2	Edmund	PRODUCT	Objects, vehicles, foods, etc. (not services)
7	2	Sisters	PERSON	People, including fictional
8	2	Witch	ORG	Companies, agencies, institutions, etc.
9	5	five pound	QUANTITY	Measurements, as of weight or distance
10	5	only two weeks	DATE	Absolute or relative dates or periods
11	6	Fralinger's	ORG	Companies, agencies, institutions, etc.
12	8	Wheatgrass	PERSON	People, including fictional
13	10	Tequila Picante Gourmet de Inclan	ORG	Companies, agencies, institutions, etc.
14	12	Felidae Platinum	ORG	Companies, agencies, institutions, etc.

15 rows × 4 columns

In [12]:

```
# Number of entities of each type, with the most common examples
label_summary = (entities_df.groupby('Label')
                 .agg(Count=('Entity', 'size'), Unique=('Entity', 'nunique'))
                 .reindex(target_labels))
label_summary['Most common entities'] = [
    ', '.join(f"{e} ({c})" for e, c in entities_df[entities_df['Label'] == lab]['Entity']
              .value_counts().head(5).items())
    for lab in target_labels]
label_summary
```

Out [12]:

	Count	Unique	Most common entities
Label			
PERSON	4351	2077	Keurig (214), Switch (129), Bisquick (102), Newman (80), Wolfgang Puck (61)
ORG	6548	2835	Amazon (1045), Amazon.com (71), Newman (70), Starbucks (58), K-Cups (53)
DATE	3007	1261	years (102), Christmas (100), today (97), daily (82), the day (74)
MONEY	872	517	1 (26), 10 (19), 5 (19), 2 (18), 20 (17)
QUANTITY	649	419	a ton (20), 34 grams (19), 1 cup (14), 8.3 ounces (13), 12 ounce (11)
PRODUCT	244	152	Gatorade (18), Subscribe (14), Starbucks (8), CA (7), CO2 (5)

6 rows × 3 columns

> **Observation:** 15,671 entities were found in 5,842 reviews. ORG is the most common type, and *Amazon* is the most frequent entity (1,045 times). The small model makes mistakes on product and brand names: *Keurig* and *Bisquick* are labelled as PERSON, *Subscribe* as PRODUCT, and MONEY values are extracted without the $ sign. Correct examples include *Starbucks* (ORG), *34 grams* (QUANTITY) and *Gatorade* (PRODUCT).

In [13]:

```
# Entities found in five example reviews
example = entities_df[entities_df['Review_ID'].isin(entities_df['Review_ID'].unique()[:5])]
for rid, group in example.groupby('Review_ID'):
    print(f"Review {rid}: {reviews.loc[rid, 'Text'][:150]}...")
    print("   ->", ', '.join(f"{e} [{l}]" for e, l in zip(group['Entity'], group['Label'])))
```

Out [13]:
```
Review 0: I have bought several of the Vitality canned dog food products and have found them all to be of good
quality. The product looks more like a stew than ...
   -> Vitality [ORG], My Labrador [PERSON]
Review 1: Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure
if this was an error or if the vendor inten...
   -> Jumbo Salted [ORG]
Review 2: This is a confection that has been around a few centuries. It is a light, pillowy citrus gelatin with nuts -
in this case Filberts. And it is cut into...
   -> around a few centuries [DATE], pillowy citrus gelatin [ORG], C.S. Lewis' [ORG], Edmund [PRODUCT], Sisters
[PERSON], Witch [ORG]
Review 5: I got a wild hair for taffy and ordered this five pound bag. The taffy was all very enjoyable with many
flavors: watermelon, root beer, melon, pepperm...
   -> five pound [QUANTITY], only two weeks [DATE]
Review 6: This saltwater taffy had great flavors and was very soft and chewy. Each candy was individually wrapped
well. None of the candies were stuck together,...
   -> Fralinger's [ORG]
```

Information Extraction

Task 7: Information extraction function

In [14]:

```
def extract_information(review):
    """Return the tokens, lemmas, POS tags, named entities and entity labels of a review."""
    doc = nlp(review)
    return {
        'Tokens': [t.text for t in doc],
        'Lemmas': [t.lemma_ for t in doc],
        'POS Tags': [t.pos_ for t in doc],
        'Named Entities': [ent.text for ent in doc.ents],
        'Entity Labels': [ent.label_ for ent in doc.ents],
    }

result = extract_information("I bought 2 packs of Lipton tea on Amazon last Monday for $10.")
for key, value in result.items():
    print(f"{key:15s}: {value}")
```

Out [14]:

```
Tokens         : ['I', 'bought', '2', 'packs', 'of', 'Lipton', 'tea', 'on', 'Amazon', 'last', 'Monday', 'for', '$',
'10', '.']
Lemmas         : ['I', 'buy', '2', 'pack', 'of', 'Lipton', 'tea', 'on', 'Amazon', 'last', 'Monday', 'for', '$', '10',
'.']
POS Tags       : ['PRON', 'VERB', 'NUM', 'NOUN', 'ADP', 'PROPN', 'NOUN', 'ADP', 'PROPN', 'ADJ', 'PROPN', 'ADP', 'SYM',
'NUM', 'PUNCT']
Named Entities : ['2', 'Lipton', 'Amazon', 'last Monday', '10']
Entity Labels  : ['CARDINAL', 'ORG', 'ORG', 'DATE', 'MONEY']
```

Task 8: Test the function using new customer reviews

In [15]:

```
new_reviews = [
    "I ordered three boxes of Nature Valley granola bars from Amazon last week for $15 "
    "and they arrived in two days.",
    "My son Rahul loves the Cadbury Dairy Milk chocolate. "
    "We finished a 500 gram pack in one weekend!",
    "The Starbucks coffee beans were stale when they arrived on 12 March, "
    "so I asked for a refund of $25.",
    "Kirkland almonds from Costco taste fresh. "
    "I eat about 30 grams every morning before going to work.",
]

extracted = [extract_information(text) for text in new_reviews]
for n, (text, info) in enumerate(zip(new_reviews, extracted), start=1):
    print(f"New Review {n}: {text}")
    print(f"   Tokens         : {info['Tokens']}")
    print(f"   Lemmas         : {info['Lemmas']}")
    print(f"   POS tags       : {info['POS Tags']}")
    print(f"   Named entities : {info['Named Entities']}")
    print(f"   Entity labels  : {info['Entity Labels']}")
    print()
```

Out [15]:

```
New Review 1: I ordered three boxes of Nature Valley granola bars from Amazon last week for $15 and they arrived in
two days.
   Tokens         : ['I', 'ordered', 'three', 'boxes', 'of', 'Nature', 'Valley', 'granola', 'bars', 'from', 'Amazon',
'last', 'week', 'for', '$', '15', 'and', 'they', 'arrived', 'in', 'two', 'days', '.']
   Lemmas         : ['I', 'order', 'three', 'box', 'of', 'Nature', 'Valley', 'granola', 'bar', 'from', 'Amazon',
'last', 'week', 'for', '$', '15', 'and', 'they', 'arrive', 'in', 'two', 'day', '.']
   POS tags       : ['PRON', 'VERB', 'NUM', 'NOUN', 'ADP', 'PROPN', 'PROPN', 'NOUN', 'NOUN', 'ADP', 'PROPN', 'ADJ',
'NOUN', 'ADP', 'SYM', 'NUM', 'CCONJ', 'PRON', 'VERB', 'ADP', 'NUM', 'NOUN', 'PUNCT']
   Named entities : ['three', 'Nature Valley', 'Amazon', 'last week', '15', 'two days']
   Entity labels  : ['CARDINAL', 'LOC', 'ORG', 'DATE', 'MONEY', 'DATE']

New Review 2: My son Rahul loves the Cadbury Dairy Milk chocolate. We finished a 500 gram pack in one weekend!
   Tokens         : ['My', 'son', 'Rahul', 'loves', 'the', 'Cadbury', 'Dairy', 'Milk', 'chocolate', '.', 'We',
'finished', 'a', '500', 'gram', 'pack', 'in', 'one', 'weekend', '!']
   Lemmas         : ['my', 'son', 'Rahul', 'love', 'the', 'Cadbury', 'Dairy', 'Milk', 'chocolate', '.', 'we',
'finish', 'a', '500', 'gram', 'pack', 'in', 'one', 'weekend', '!']
   POS tags       : ['PRON', 'NOUN', 'PROPN', 'VERB', 'DET', 'PROPN', 'PROPN', 'PROPN', 'NOUN', 'PUNCT', 'PRON',
'VERB', 'DET', 'NUM', 'NOUN', 'NOUN', 'ADP', 'NUM', 'NOUN', 'PUNCT']
   Named entities : ['500 gram', 'one weekend']
   Entity labels  : ['QUANTITY', 'DATE']

New Review 3: The Starbucks coffee beans were stale when they arrived on 12 March, so I asked for a refund of $25.
   Tokens         : ['The', 'Starbucks', 'coffee', 'beans', 'were', 'stale', 'when', 'they', 'arrived', 'on', '12',
'March', ',', 'so', 'I', 'asked', 'for', 'a', 'refund', 'of', '$', '25', '.']
   Lemmas         : ['the', 'Starbucks', 'coffee', 'bean', 'be', 'stale', 'when', 'they', 'arrive', 'on', '12',
'March', ',', 'so', 'I', 'ask', 'for', 'a', 'refund', 'of', '$', '25', '.']
   POS tags       : ['DET', 'PROPN', 'NOUN', 'NOUN', 'AUX', 'ADJ', 'SCONJ', 'PRON', 'VERB', 'ADP', 'NUM', 'PROPN',
'PUNCT', 'ADV', 'PRON', 'VERB', 'ADP', 'DET', 'NOUN', 'ADP', 'SYM', 'NUM', 'PUNCT']
   Named entities : ['Starbucks', '12 March', '25']
   Entity labels  : ['ORG', 'DATE', 'MONEY']

New Review 4: Kirkland almonds from Costco taste fresh. I eat about 30 grams every morning before going to work.
   Tokens         : ['Kirkland', 'almonds', 'from', 'Costco', 'taste', 'fresh', '.', 'I', 'eat', 'about', '30',
'grams', 'every', 'morning', 'before', 'going', 'to', 'work', '.']
   Lemmas         : ['Kirkland', 'almond', 'from', 'Costco', 'taste', 'fresh', '.', 'I', 'eat', 'about', '30', 'gram',
'every', 'morning', 'before', 'go', 'to', 'work', '.']
   POS tags       : ['PROPN', 'NOUN', 'ADP', 'PROPN', 'NOUN', 'ADJ', 'PUNCT', 'PRON', 'VERB', 'ADV', 'NUM', 'NOUN',
'DET', 'NOUN', 'ADP', 'VERB', 'PART', 'VERB', 'PUNCT']
   Named entities : ['Kirkland', 'Costco', 'about 30 grams every morning']
   Entity labels  : ['ORG', 'ORG', 'QUANTITY']
```

Task 9: Display the extracted information in a structured format

In [16]:

```
# Summary table: one row per review
summary_rows = []
for n, (text, info) in enumerate(zip(new_reviews, extracted), start=1):
    pos_count = Counter(info['POS Tags'])
    summary_rows.append({
        'Review': n,
        'Tokens': len(info['Tokens']),
        'Nouns': pos_count['NOUN'] + pos_count['PROPN'],
        'Verbs': pos_count['VERB'],
        'Adjectives': pos_count['ADJ'],
        'Adverbs': pos_count['ADV'],
        'Named Entities': '; '.join(f"{e} ({l})" for e, l in
                                    zip(info['Named Entities'], info['Entity Labels'])),
    })
pd.DataFrame(summary_rows).set_index('Review')
```

Out [16]:

Columns 1–5 of 6

Review	Tokens	Nouns	Verbs	Adjectives	Adverbs
1	23	8	2	1	0
2	20	9	2	0	0
3	23	5	2	1	1
4	19	6	3	1	1

Columns 6–6 of 6

Review	Named Entities
1	three (CARDINAL); Nature Valley (LOC); Amazon (ORG); last week (DATE); 15 (MONEY); two days (DATE)
2	500 gram (QUANTITY); one weekend (DATE)
3	Starbucks (ORG); 12 March (DATE); 25 (MONEY)
4	Kirkland (ORG); Costco (ORG); about 30 grams every morning (QUANTITY)

4 rows × 6 columns

In [17]:

```
# Entity table: one row per extracted entity
entity_table = pd.DataFrame([
    {'Review': n, 'Entity': e, 'Label': l, 'Description': spacy.explain(l)}
    for n, info in enumerate(extracted, start=1)
    for e, l in zip(info['Named Entities'], info['Entity Labels'])])
entity_table
```

Out [17]:

	Review	Entity	Label	Description
0	1	three	CARDINAL	Numerals that do not fall under another type
1	1	Nature Valley	LOC	Non-GPE locations, mountain ranges, bodies of water
2	1	Amazon	ORG	Companies, agencies, institutions, etc.
3	1	last week	DATE	Absolute or relative dates or periods
4	1	15	MONEY	Monetary values, including unit
5	1	two days	DATE	Absolute or relative dates or periods
6	2	500 gram	QUANTITY	Measurements, as of weight or distance
7	2	one weekend	DATE	Absolute or relative dates or periods
8	3	Starbucks	ORG	Companies, agencies, institutions, etc.
9	3	12 March	DATE	Absolute or relative dates or periods
10	3	25	MONEY	Monetary values, including unit
11	4	Kirkland	ORG	Companies, agencies, institutions, etc.
12	4	Costco	ORG	Companies, agencies, institutions, etc.
13	4	about 30 grams every morning	QUANTITY	Measurements, as of weight or distance

14 rows × 4 columns

In [18]:

```
# Token-level table for the first new review
info = extracted[0]
pd.DataFrame({'Token': info['Tokens'], 'Lemma': info['Lemmas'], 'POS Tag': info['POS Tags']},
             index=range(1, len(info['Tokens']) + 1)).T
```

Out [18]:

Columns 1–17 of 23

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Token	I	ordered	three	boxes	of	Nature	Valley	granola	bars	from	Amazon	last	week	for	$	15	and
Lemma	I	order	three	box	of	Nature	Valley	granola	bar	from	Amazon	last	week	for	$	15	and
POS Tag	PRON	VERB	NUM	NOUN	ADP	PROPN	PROPN	NOUN	NOUN	ADP	PROPN	ADJ	NOUN	ADP	SYM	NUM	CCONJ

Columns 18–23 of 23

	18	19	20	21	22	23
Token	they	arrived	in	two	days	.
Lemma	they	arrive	in	two	day	.
POS Tag	PRON	VERB	ADP	NUM	NOUN	PUNCT

3 rows × 23 columns

> **Observation:** The function correctly extracts most of the information: *Amazon*, *Starbucks*, *Costco* and *Kirkland* as organisations, *last week* and *12 March* as dates, *$15* and *$25* as money, and *500 gram* as a quantity. It also makes mistakes: *Nature Valley* (a brand) is labelled as a location, the person *Rahul* and the product *Cadbury Dairy Milk* are not detected, *about 30 grams every morning* combines a quantity with a time, and *taste* in review 4 is tagged as a noun instead of a verb. A larger spaCy model (such as en_core_web_lg) or a model trained on product reviews would give more accurate results.

Result

Thus the customer reviews were processed using the spaCy en_core_web_sm pipeline. Tokens, lemmas and POS tags were extracted, the POS categories were identified and counted (nouns were the most frequent at 16.14%), and Named Entity Recognition extracted 15,671 PERSON, ORG, DATE, MONEY, QUANTITY and PRODUCT entities into a structured DataFrame. An information extraction function was created and tested on four new customer reviews, and its output was displayed in structured tables.